

# Relazione Piattaforme Software Beeskium

Davide De Maria - 255296

Paolo Ferlaino - 251729

A.A. 2025/2026

# 1 Introduzione (0.5 Pg)

Beeskium è una webapp concepita e sviluppata per ottimizzare la gestione operativa e amministrativa di una barberia moderna.

A differenza dei software commerciali generici, il progetto si presenta molto più snello e diretto, senza confondere l'utente e rendendo l'esperienza fluida e veloce (come una normale prenotazione dovrebbe essere).

L'architettura dell'applicazione è stata strutturata per garantire scalabilità e manutenibilità grazie ai potenti software di Spring e Angular la modularizzazione rende l'ampliamento della piattaforma semplicissimo.

Dal punto di vista funzionale, il core di Beeskium si articola su tre moduli principali:

- **Gestione dei Flussi e dei Turni:** Un motore di pianificazione algoritmica che previene i conflitti di prenotazione e ottimizza i tempi di attesa.
- **Anagrafica e CRM:** Un database relazionale per la gestione dei profili clienti, dello storico dei trattamenti e delle preferenze individuali.
- **Creazione Filiali:** Un pannello di controllo per la creazione di nuove filiali della catena.

Sotto il profilo tecnico, lo sviluppo ha seguito i moderni standard del web lato fullstack.

Il backend gestisce la logica di business, l'autenticazione sicura degli utenti e l'integrità dei dati tramite un database strutturato e per coerenza è separato progettualmente dal frontend (come si vede dalle repo).

Il frontend (che gira concettualmente su un altro server), d'altra parte, è stato progettato come una Single Page Application (SPA Angular) focalizzata sull'efficienza dell'interfaccia utente (UI), garantendo tempi di risposta minimi e una navigazione fluida su mobile.

## 2 Analisi dei requisiti (0.5 Pg)

L'analisi dei requisiti definisce le specifiche funzionali e i vincoli tecnologici necessari allo sviluppo della piattaforma *Beeskium*, tracciando gli obiettivi del backend (Spring) e del frontend (Angular). L'applicazione inoltre si basa sulla completa flessibilità riguardo le operazioni CRUD.

### 2.1 Requisiti Funzionali e Attori

Il sistema prevede due attori principali, ognuno con permessi e interazioni distinte all'interno dell'applicazione:

- **Cliente:** Registrazione/Login, visualizzazione delle filiali e dei servizi, prenotazione degli appuntamenti in tempo reale secondo disponibilità.
- **Admin:** Creazione e gestione delle filiali (orari di apertura/giorni feriali) e privilegi massimi sugli appuntamenti.

### 2.2 Requisiti Non Funzionali (RNF)

Per garantire la scalabilità e la sicurezza del software, sono stati identificati i seguenti requisiti non funzionali:

|                         |                                                                                                                            |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Sicurezza</b>        | Autenticazione <i>stateless</i> tramite Token JWT gestiti lato Spring Security. Password 'hashate' per maggiore sicurezza. |
| <b>Disaccoppiamento</b> | Architettura RESTful pulita: il backend espone solo API, il frontend Angular si occupa del rendering logico.               |
| <b>Reattività</b>       | Interfaccia Angular progettata e studiata con focus sulla piattaforma più usata dall'utente medio (mobile).                |
| <b>Robustezza</b>       | Gestione transazionale delle prenotazioni per evitare corruzione dei dati in caso di richieste concorrenti.                |

## 3 Back-end (1-2 Pg)

Il backend è il cuore pulsante del progetto, scritto in Java Spring per rendere il codice leggibile e strutturato, in particolare Spring Boot per i suoi vantaggi in termini di configurazione iniziale, si comporta come una API RESTful a tutti gli effetti; architettato a livelli utilizzando lo schema strutturale classico (*Controller* → *Service* → *Repository* → *Model*).

### 3.1 Controller

La struttura del controller, costruita per essere minimal e semplice da usare, è stata architettata mediante un approccio bottom-up, la struttura è nata in seguito alla costruzione del **Diagramma ER** (che verrà presentato in seguito) per rispondere a tutte le esigenze del cliente (operazioni CRUD di base) nel modo più semplice e completo possibile. Essenziale inoltre il lavoro dei DTO riguardo lo scambio di informazioni tra client e server, locati in una cartella dedicata permettono di sfruttare tutto il potenziale di java con annotazioni del tipo *@RequestBody*.

Tutti i metodi sono stati testati con l'ausilio di *Postman*, software potentissimo per le nostre esigenze.

Di seguito verranno trattati tutti i principali *endpoint* REST, per facilità di comprensione sono scritti in questa forma [METODO HTTP/metodo java(parametri)]:

#### 3.1.1 POST/reserve(reservationDate, staffId, serviceIds)

Metodo fulcro di questo backend, come dice lo stesso titolo riceve prenotazioni e le trasferisce al *Service* per interpretarle e validarle, è un metodo post, molto più gestibile e adatto al contesto (secondo lo standard CRUD per il rest), difatti inserisce un nuovo record nella tabella del database (in caso di input valido). Prende come parametri un timestamp (LocalDateTime), l'identificativo dello staff scelto e una lista di IDs per ogni servizio disponibile nella filiale scelta.

#### 3.1.2 GET/reservations()

Metodo molto autoesplicativo, permette all'utente loggato (essenziale per il 90% dei metodi tranne login e register) di richiedere le sue prenotazioni effettuate, restituite in formato rigorosamente JSON come da standard.

### 3.1.3 GET/services()

Metodo inizialmente creato per ottenere i servizi disponibili nella barberia (è nato come un progetto di una singola filiale), è un perfetto esempio della scalabilità e della potenza di Spring, infatti è stato successivamente modificato e sostituito per accogliere la presenza di più filiali senza alcuno sforzo.

### 3.1.4 DELETE/deleteReservation(reservationId)

Anch'esso ottimo esempio, difatti non è stato necessario rimpiazzato e la logica dichiarata in precedenza era già ottima per un sistema multi-filiale. Cancella la prenotazione dato un ID in input.

### 3.1.5 GET/availabilities(barberShopId, staffId, serviceIds)

Metodo clue del nostro progetto, calcola a intervalli di 15 minuti (modificabili tramite un parametro) gli slot orari disponibili per la prenotazione evitando sovrapposizioni e facilitando la vita al personale, l'utente può scegliere la fascia oraria facilmente cliccando dei pulsanti con gli orari. Anch'esso scalato molto facilmente per un approccio multi-barbershop.

### 3.1.6 POST/login - POST/register

Metodi di base per la registrazione e login mediante token JWT, abbiamo preferito questo approccio a sistemi più moderni per una questione propedeutica e per comprendere meglio come funzionano alla base i sistemi moderni.

## 3.2 Service

Il service implementa la logica di backend, senza entrare troppo nel lato algoritmico presente nel codice, sono degni di nota i filtri e le validazioni lato register e login.

La gestione dei token JWT che permettono di creare una sessione univoca con l'utente e abilitano l'accesso a tutti gli altri endpoint (come reserve/reservations).

Il controllo dei privilegi e la gestione delle operazioni CRUD da parte degli admin.

L'algoritmo dietro la creazione di una prenotazione e la validazione degli orari per non incorrere in sovrapposizioni.

### 3.3 Repository & Model

La parte più ragionata e meticolosa del nostro progetto è proprio il model (accompagnato da Repository per le operazioni sul db). Come detto in precedenza è figlio di un diagramma ER studiato e successivamente portato in digitale per esporlo nella relazione, il focus durante la stesura del diagramma era solo la semplicità, infatti la complessità dei sistemi legacy e il cosiddetto *spaghetti code* sono stati il motivo per cui abbiamo deciso questo approccio minimal ma completo (Ovviamente la complessità è presente ma le fondamenta a parer nostro devono essere semplici e solide). Il progetto ha poi subito un'evoluzione per supportare l'approccio a *catena* (tipo mcdonalds), e abbiamo potuto sperimentare la solidità delle nostre scelte proprio in questa fase, difatti il sistema è stato ampliato senza modificare (se non impercettibilmente) le fondamenta create e con una semplicità ottime. Di seguito è allegata la struttura di base del nostro diagramma ER, successivamente ampliato e perfezionato.

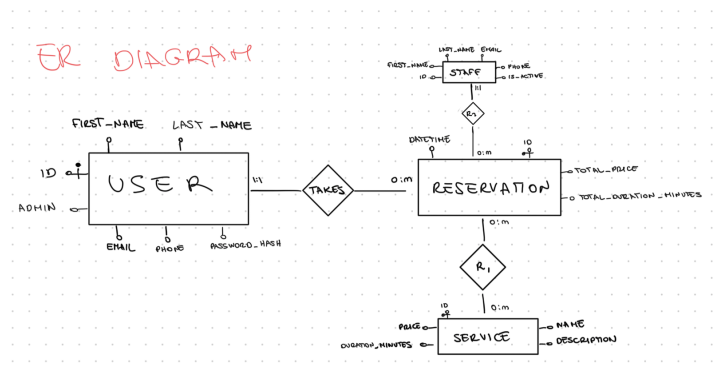
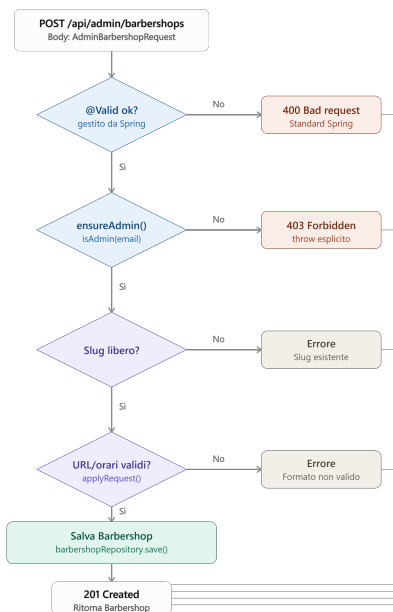
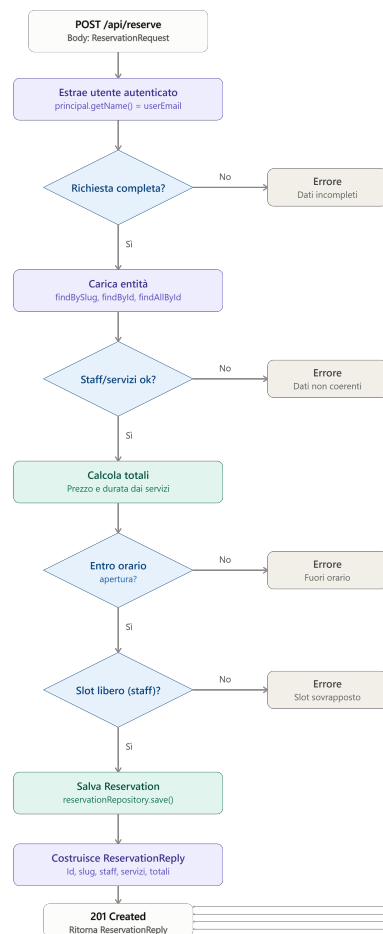


Figura 1: ER Diagram



(a) Creation Flow Diagram



(b) Presentation Flow Diagram

Figura 2: Diagrammi di flusso

## 4 Front-end (1-2 Pg)

### 4.1 Introduzione e Scelte Tecnologiche

Il frontend dell'applicazione Beeskium è interamente realizzato con **Angular**, un framework *component-based* guidato da **TypeScript**. A differenza dei tradizionali approcci server-side, Angular adotta un'architettura *Client-Side Rendering* (CSR) per *Single Page Application* (SPA).

La logica di presentazione, la gestione dello stato e il routing vengono eseguiti direttamente nel browser dell'utente. Il web server Beeskium ospita e distribuisce file statici compilati (HTML, CSS, JavaScript), mentre il frontend comunica in modo asincrono con il backend tramite **API REST**. L'uso di TypeScript introduce una forte tipizzazione, riducendo gli errori a runtime e migliorando la manutenibilità del codice a lungo termine.

### 4.2 Struttura delle Cartelle e Componenti

L'organizzazione dei file segue le *best practice* di Angular, separando le responsabilità in moduli logici ben definiti:

**Components:** Ospita blocchi grafici atomici e riutilizzabili. Gli esempi principali sono **reservation-box** (per la card riassuntiva delle prenotazioni) e **service-box** (per i singoli trattamenti del salone). Ogni componente è diviso in tre file: `.html` per il template dichiarativo, `.css` per gli stili incapsulati e `.ts` per la classe logica e i decoratori `@Component`.

**Pages:** Contiene i macro-componenti associati direttamente alle rotte di navigazione (URL). Ogni cartella rappresenta una schermata del flusso utente:

- **admin-barbershop:** Pannello di gestione e configurazione per gli amministratori dei saloni.
- **barbershop-list:** Schermata principale per l'esplorazione e la ricerca dei negozi disponibili.
- **booking:** Interfaccia critica per la selezione guidata di data, ora, operatore e servizio.
- **home:** Pagina di atterraggio principale contenente le svariate barberie.
- **profile:** Area privata dello storico utente ergo le sue prenotazioni e barberie.



- **login / register:** Moduli verticali per l'autenticazione, la sicurezza e la creazione di nuovi account.

## 4.3 Strati di Controllo, Logica e Comunicazione

### 4.3.1 Sicurezza e Flussi (guards e interceptors)

- **Guards (auth.guard.ts):** Funzioni di tipo `CanActivateFn` che sfruttano la *Dependency Injection* tramite la funzione `inject()` per controllare la navigazione. La `authGuard` protegge le rotte riservate reindirizzando i visitatori alla pagina di login tramite un `UrlTree`, mentre la `guestGuard` opera al contrario, impedendo agli utenti già autenticati di tornare alle schermate di login o registrazione.
- **Interceptors (auth.interceptor.ts):** Sfrutta l'interfaccia funzionale `HttpInterceptorFn` per manipolare centralmente le richieste HTTP in uscita. Se presente, preleva il token JWT dall'`AuthService` e inietta l'header `Authorization: Bearer <token>`. Utilizza l'operatore `RxJS catchError` per intercettare gli errori HTTP 401 (*Unauthorized*): se l'errore proviene da un endpoint privato, invoca automaticamente il metodo di `logout` per invalidare la sessione scaduta sul client.

### 4.3.2 Logica di Business e API (services)

Classi decorate con `@Injectable({ providedIn: 'root' })` che fungono da istanze *singleton* per la gestione dello stato e delle chiamate HTTP tramite il modulo `HttpClient` e la libreria reattiva **RxJS**:

- **auth.service.ts:** Cuore della gestione sessioni. Integra i moderni *Signals* di Angular (`currentUser`, `isAuthenticated`) per uno stato globale reattivo. Salva in modo persistente il token e i dati utente nel `localStorage` e convalida la sessione all'avvio dell'app tramite l'endpoint `/api/auth/me`.
- **admin.service.ts:** Espone i metodi CRUD (`GET`, `POST`, `PUT`, `DELETE`) verso gli endpoint protetti `/api/admin/...` per la configurazione dei saloni, dei listini dei servizi e delle anagrafiche dello staff.
- **barbershop.service.ts:** Recupera i dati pubblici dei saloni. Implementa logiche di *fallback* con costanti locali mock (`BARBERSHOPS`, `STAFF_MEMBERS`) in caso di errore di rete e funzioni helper (`toBarbershop`).

- `reservation.service.ts`: Gestisce i flussi di prenotazione ed eliminazione degli appuntamenti. Include il metodo `getAvailabilities` per richiedere gli slot liberi compilando dinamicamente i parametri di query (`HttpParams`) per i servizi selezionati.
- `services.service.ts`: Fornisce l'elenco dei trattamenti commerciali, sia globali che filtrati per singolo salone.

#### 4.3.3 Utilità e Data Transformation (utils)

- `opening-hours.ts`: Contiene la logica pura `toMinutes` per convertire stringhe relative all'orario impostato della prenotazione in minuti totali trascorsi dalla mezzanotte ed esegue validazioni matematiche strette. La funzione `isOpenForSchedule` sfrutta questi dati per calcolare all'istante se il salone è aperto e disponibile in base all'orario di sistema.
- `reservation-mapper.ts`: Agisce come *Anti-Corruption Layer* tra il backend e l'interfaccia. La funzione `mapReservationReply` trasforma i DTO grezzi (`ReservationReply`) in oggetti strutturati per il client (`Reservation`), risolvendo le relazioni tra gli ID dei servizi/staff e i rispettivi nomi descrittivi leggibili dall'utente.

### 4.4 Flusso dei Dati ed Esperienza Utente (UX)

L'architettura client-side di Angular garantisce una reattività immediata del frontend Beeskium. Quando l'utente seleziona un elemento grafico o cambia pagina, il sistema di routing intercetta l'azione e aggiorna la vista senza ricaricare il browser.

Il componente attivo interroga il relativo servizio, il quale effettua una chiamata asincrona HTTP verso il server. Durante l'attesa del payload, l'interfaccia mostra indicatori di caricamento locali senza bloccare l'interazione complessiva (**UI non-bloccante**). Alla ricezione dei dati, i *Signals* notificano i template HTML che aggiornano istantaneamente la visualizzazione tramite *data binding*.

## 5 Considerazioni finali (0.5 Pg)

L'applicazione "Beeskium" costituisce un esempio completo e ben strutturato di sistema web dedicato alla gestione delle prenotazioni per barberie moderne. L'adozione di un'architettura a livelli e la gestione accurata delle entità (come clienti, barbieri, servizi e appuntamenti) contribuiscono a rendere il progetto estendibile e robusto. Principale punto di forza dell'applicazione è rappresentato dalla netta separazione delle responsabilità. Trattandosi di un'architettura basata su Web Services, l'integrazione tra il backend sviluppato in Spring Boot e il frontend realizzato in Angular rappresenta una scelta strategica e moderna. Questo approccio disaccoppiato non solo mantiene il codice pulito e organizzato, ma garantisce anche un'interfaccia utente altamente dinamica e reattiva, fondamentale per ciascun servizio al passo coi tempi. Pur essendo un progetto nato con finalità didattiche, l'applicazione possiede caratteristiche che la rendono perfettamente adatta a scenari commerciali reali. Con opportuni miglioramenti futuri, il sistema potrebbe gestire moli di lavoro ben più complesse.

I principali ambiti di sviluppo includono:

- Gestione in-site delle transazioni monetarie ai saloni ed eventualmente ai barbieri stessi
- Il potenziamento della sicurezza (es. gestione avanzata dei ruoli cliente/amministratore e protezione dei dati personali).
- L'ulteriore ottimizzazione dell'interfaccia utente (UI/UX) in Angular.

In conclusione, il progetto dimostra come sia possibile sviluppare piattaforme gestionali moderne e modulari combinando la potenza di Spring Boot per i servizi di backend e la versatilità di Angular per il frontend. L'analisi evidenzia l'importanza di una progettazione strutturata per garantire un software scalabile, manutenibile e di alta qualità nel settore dei servizi estetici.